

Object Oriented Programming Lab

(CL1004)

Date: May 21, 2026

Instructor(s)

Mr. Ahmad Jawad Mustasim, Mr. Durraiz Waseem,
Ms. Eisha Khan, Ms. Mamoon Akbar,
Mr. M. Jawad Khan, Mr. M. Nouman Hanif,
Ms. Zoha Waheed

Final Exam**Total Time:** 2h 20m**Total Marks:** 45**Total Questions:** 2
Roll No
Section
Student Signature

Do not write below this line

Submission Path**Windows + R:** \\Exam\ Exam\ Final Exam\ OOP Lab\ [your section]\ Submissions**Important Instructions**

- Duration of exam is **2 hours**. An **additional 20 minutes** have been allocated for proper submission of your files. Therefore, stop attempting the exam once the 2 hour duration is over and use the remaining time to **carefully** submit your file.
- Make a **.cpp** file for each question and name it using your roll number e.g. **25L-1234-q1.cpp**
- Place all the exam (.cpp) files in a **single folder** and **compress** the folder into a **ZIP file** named using your roll number e.g. **25L-1234.zip** and upload the ZIP file to the designated submission folder.
- After submitting, **verify** your zip file is visible in the **submission folder of your section**, and **check that a file size is shown** next to it (to ensure it is not 0 KB file which indicates the file is empty or corrupted).
- To ensure everything is properly submitted, you may download the uploaded ZIP file and check its contents. It is your responsibility to save and submit all required files in the correct format and ensure that the submitted files are complete and not corrupted.

Attempt all the questions.**CLO #3****Q1: Punjab Inter-City Transport Management System [25 marks]****Instructions:**

- You may not use any STL containers (e.g. vector, map) or built-in string manipulation functions.
- Use dynamic memory wherever appropriate. Global variables not allowed
- Use Inheritance /Composition/ Aggregation (whichever is applicable)
- Use char pointers as data members instead of strings (if applicable).

The Government of Punjab is initiating a **Public Transport Digitalization Project** to improve the management of transport routes, drivers and buses. You are hired to develop a C++ application that manages this system using object-oriented principles.

1. System Requirements:

[1.5 marks]

• **Route**

- Route Code
- Departure City
- Arrival City
- Departure Timing
- Distance in KMs

[1.5 marks]

• **Driver**

- Name
- CNIC
- License Number

[2 marks]

• **Bus**

- Bus Registration Number
- Chassis Number
- Passenger Capacity
- Assigned to a Route *association*
- Assigned to a Driver

2. Functionalities to implement:

- Constructors for all classes (default, parameterized, copy) [3+3+3 = 9 marks]
- Destructors for all classes [1+1+1 = 3 marks]
- Input validations where applicable [2 marks]
- Implement == operator in the Route class. Two routes are considered equal if both their departure cities and arrival cities match (case-sensitive comparison). [2 marks]
- Assign a route and driver to a bus. [2 marks]
- Display complete details of buses along with their assigned driver and route information. [2 marks]
- You must demonstrate the **creation of multiple Bus, Driver and Route objects in your main code** (using dynamic memory). Use **dummy test data** directly in your code instead of prompting the user for input. **Release all dynamically allocated memory** before program termination.

CLO #2**Q2: Military Aircraft Management System [20 marks]**

You are tasked with designing a software system that models different categories of military aircraft used in PAF operations such as **Fighter Jet, AWACS, Trainer and Bomber**. Your design must reflect strong object-oriented principles such as inheritance, polymorphism, aggregation/composition and templates.

1. Create an abstract base class **WarPlane** with the following: [3 marks]
- A protected string data member `planeName`
 - `void description()`: A pure virtual function that should display plane's name, role (as a descriptive message defined in each derived class) and engine information (to be added in part-3).
 - A destructor for proper memory cleanup.

2. Derive the following concrete classes: [8 marks]
- `FighterJet`
 - `AWACS` (Airborne Warning and Control System)
 - `Trainer`
 - `Bomber`

Each derived class must print plane name, role message, and engine details inside its `description()` function. For example:

- `FighterJet`: "Engaging enemy aircraft and maintaining air superiority."
- `AWACS`: "Providing airborne surveillance and battlefield coordination."
- `Trainer`: "Training new fighter pilots for advanced missions."
- `Bomber`: "Carrying out long range strategic strike missions."

3. Define a class `Engine` representing aircraft engine details: [4 marks]
- string type (e.g. "Turbofan", "Turbojet", "Piston")
 - int thrust (kiloNewtons kN)

Each war plane must be initialized with its specific engine during construction time. **Every WarPlane contains an Engine as an internal component.** Once the plane is created, engine details must remain constant throughout the object's lifetime. Use appropriate relationship to ensure the engine is tied to the plane it belongs to.

Update the **constructors** of all warplane types to accept engine specifications and store them and enhance the **description()** function of each warplane class to additionally display its **engine type** and **thrust** in a meaningful way

4. In `main()`, use a **dynamic array** (of size 4) of **pointers to WarPlane** [3 marks]
- Add planes dynamically (use hardcoded test data for plane type and engine info)
 - Call `description` for all planes
 - Memory management must be handled properly.
5. Write a **templated function** `analyzeAircraft(T& plane)` that accepts any type of plane (`FighterJet`, `Bomber` etc.) and displays: [2 marks]

"Running diagnostics on [plane's name]. It is used for [plane's description]"

Good luck! May your Final score soar to a legendary 6-0, with no code crashes